Title:

AI Developer Copilot: A BERT and RAG-Based Code Intelligence System

Section : 4

Team Number : 7

Abstract :

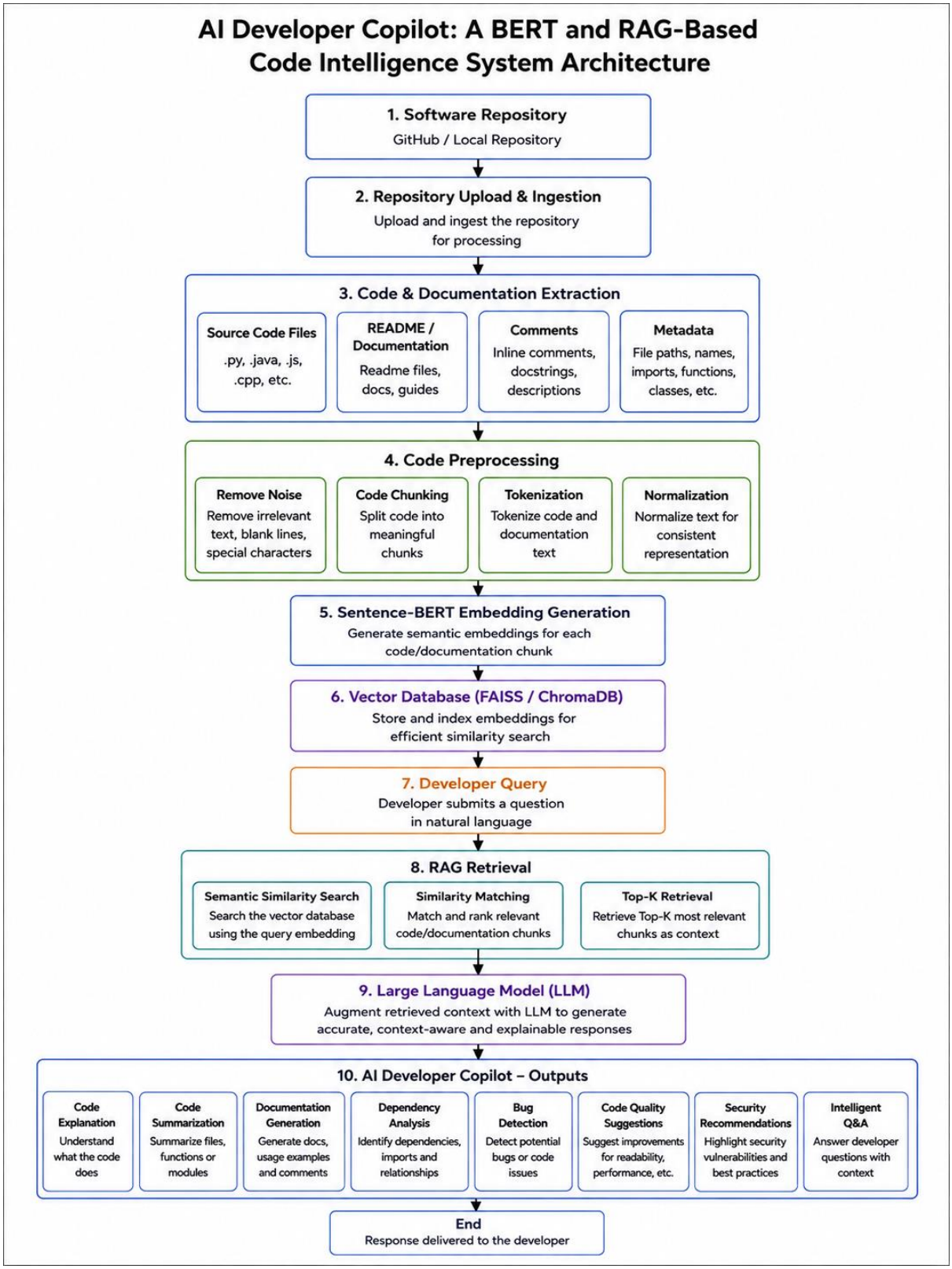
Modern software projects often consist of large and complex codebases, making it difficult for developers to understand project structures, locate relevant code, and maintain applications efficiently. Traditional code search tools provide limited contextual understanding and often fail to deliver intelligent, project-specific assistance. This project proposes an **AI Developer Copilot** that combines **BERT-based semantic embeddings** with **Retrieval-Augmented Generation (RAG)** to retrieve relevant code snippets and generate accurate, context-aware explanations for developer queries.

The proposed system preprocesses source code, generates semantic embeddings, stores them in a vector database, and uses a Large Language Model (LLM) to provide code explanations, summarization, dependency analysis, documentation support, and code quality recommendations. The system is evaluated using metrics such as retrieval accuracy, Precision@K, Recall@K, response relevance, and user satisfaction. The resulting framework aims to improve developer productivity, accelerate onboarding, simplify software maintenance, and deliver reliable, explainable, and intelligent code assistance.

Datasets :

Dataset	URL Link
CodeSearchNet	https://github.com/github/CodeSearchNet
The Stack (Optional)	https://huggingface.co/datasets/bigcode/the-stack
HumanEval (Evaluation)	https://github.com/openai/human-eval

Approach :


AI Developer Copilot: A BERT and RAG-Based Code Intelligence System Architecture
1. Software Repository
GitHub / Local Repository
2. Repository Upload & Ingestion
Upload and ingest the repository for processing
3. Code & Documentation Extraction
Source Code Files
.py, .java, .js, .cpp, etc.
README / Documentation
Readme files, docs, guides
Comments
Inline comments, docstrings, descriptions
Metadata
File paths, names, imports, functions, classes, etc.
4. Code Preprocessing
Remove Noise
Remove irrelevant text, blank lines, special characters
Code Chunking
Split code into meaningful chunks
Tokenization
Tokenize code and documentation text
Normalization
Normalize text for consistent representation
5. Sentence-BERT Embedding Generation
Generate semantic embeddings for each code/documentation chunk
6. Vector Database (FAISS / ChromaDB)
Store and index embeddings for efficient similarity search
7. Developer Query
Developer submits a question in natural language
8. RAG Retrieval
Semantic Similarity Search
Search the vector database using the query embedding
Similarity Matching
Match and rank relevant code/documentation chunks
Top-K Retrieval
Retrieve Top-K most relevant chunks as context
9. Large Language Model (LLM)
Augment retrieved context with LLM to generate accurate, context-aware and explainable responses
10. AI Developer Copilot – Outputs
Code Explanation
Understand what the code does
Code Summarization
Summarize files, functions or modules
Documentation Generation
Generate docs, usage examples and comments
Dependency Analysis
Identify dependencies, imports and relationships
Bug Detection
Detect potential bugs or code issues
Code Quality Suggestions
Suggest improvements for readability, performance, etc.
Security Recommendations
Highlight security vulnerabilities and best practices
Intelligent Q&A
Answer developer questions with context
End
Response delivered to the developer

Project Group Members

S.No.	Roll No.	Student Name
1	2420030367	Sarvani Topella
2	2420030507	Tanvi Bandaru
3	2420090112	Amy Jamana